

Data & AI

8. Your Own Image Classifier

Dasaem Jeong · Art & Technology, Sogang University

Last Time

A classifier trained on somebody else's data

- 784 numbers in, 10 numbers out
- 50,890 parameters, trained in about a second
- Training accuracy 99.2%, test accuracy 95.0%

Today

- Image classification, and training in batches
- Assignment 1: an image classifier on images you collect
- Git, GitHub and GitHub Pages, because that is how you hand it in
- Google Colab, for those without a fast computer

Image classification

Supervised Learning

Every training example is an input paired with its answer

- Last time: 784 pixel values, paired with the digit
- Assignment 1: a colour photo, paired with one of your categories
- Someone has to supply every answer

What the Model Actually Sees

the photo
2121 x 1414 pixels



what the model is given
224 x 224 pixels



- Shorter side shrunk to 256, then the centre 224 x 224 cut out
- Every photo, whatever its size, becomes the same 224 x 224

A Colour Image as Numbers

224 x 224 pixels



red: 224 x 224 numbers



green: 224 x 224 numbers



blue: 224 x 224 numbers



- Three numbers per pixel: red, green and blue
- $224 \times 224 \times 3 = 150,528$ numbers for one photo

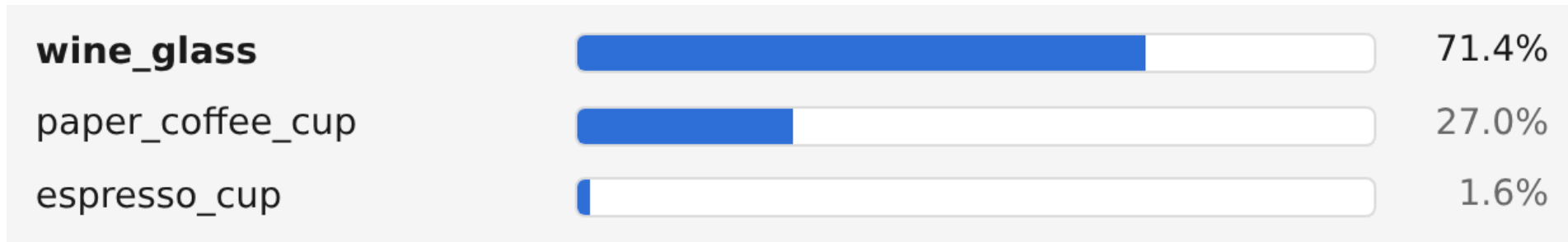
From a Photo to an Answer

The same steps as last time

1. 150,528 numbers in
2. One score per category out, 3 to 5 of them
3. Softmax turns the scores into probabilities
4. The category with the highest probability is the answer

Loss for One Photo

loss = $-\log p$, where p is the probability given to the right category



- If the right answer is wine_glass: $-\log 0.714 = 0.34$
- If it were espresso_cup: $-\log 0.016 = 4.14$

Batches and stochastic gradient descent

One Step of Gradient Descent

As in lectures 4 and 6

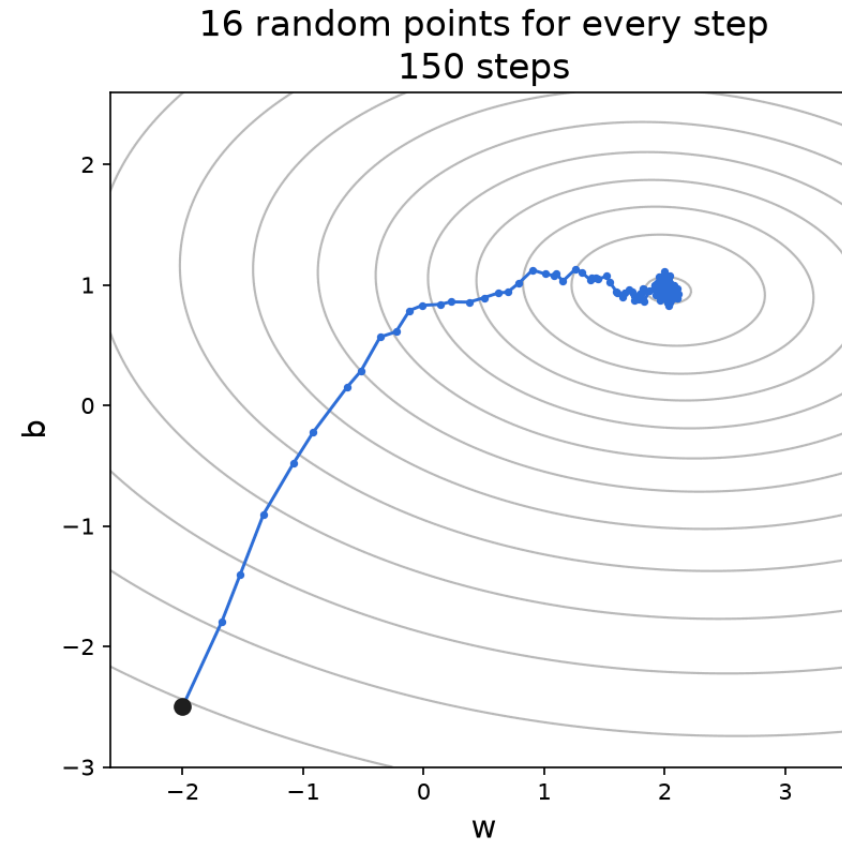
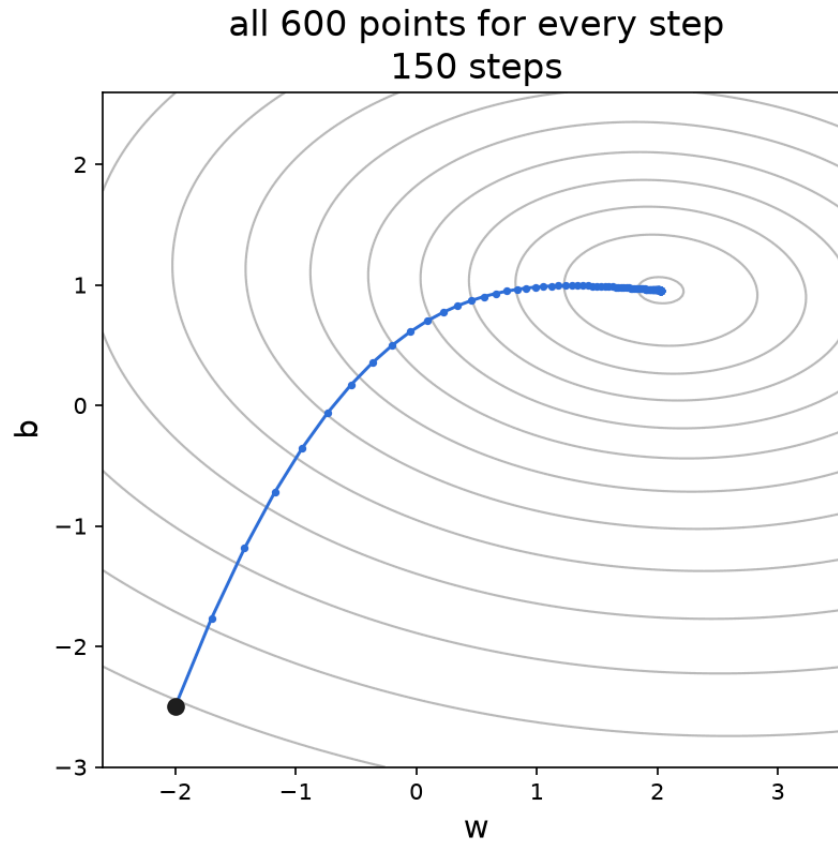
1. Run every training image forward and backward
 2. Average their slopes
 3. Move every parameter a little against its slope
- 60,000 training images: 60,000 forward and backward passes for one step

One Step from a Batch

1. Pick a few images at random, for example 32
 2. Run only those forward and backward, and average their slopes
 3. Move every parameter a little against that slope
- The batch slope is close to the full slope, but not equal to it
 - One step costs 32 images instead of 60,000

Gradient Descent and Stochastic Gradient Descent

Fitting a line $y = w x + b$ to 600 points, same learning rate, 150 steps each



- Right: each step uses 16 points, so the path wobbles but ends in the same place

Epochs and Batches in Assignment 1

450 photos, 80% for training, batches of 32

- 360 training photos: 11 batches of 32 and one of 8, so 12 steps per epoch
- 15 epochs: 180 steps
- The order is shuffled again at the start of every epoch

Choosing the Batch Size

- Smaller batches: more steps per epoch, and each step wobbles more
- Larger batches: fewer, steadier steps, and more memory on the GPU
- 32 is a common place to start

What you will build

The Demo

espresso_cup · paper_coffee_cup · wine_glass

self test passed: the browser agrees with Python (largest gap 0.0000)

Webcam

Upload a picture

Draw it

Choose File 0025.jpg

Pick a photo from your own phone or computer.



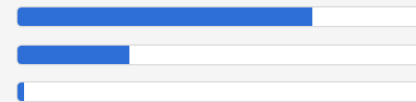
What the model actually sees



wine_glass

paper_coffee_cup

espresso_cup



71.4%

27.0%

1.6%

Three Ways to Feed It a Picture

- Your webcam
- A file you upload
- Something you draw with the mouse

It Runs Inside the Page

- No server, no Python, no installing anything
- The address keeps working after you close your laptop
- Nobody's photos are sent anywhere

Assignment 1

Train an image classifier on images you collect yourself

- 3 to 5 categories of your own choosing
- Due Friday 16 October, 22:00, on Cyber Campus (LMS)
- Everything at github.com/jdasam/mas1004-assignment1

The assignment

The Seven Problems

1. Choose your categories and collect the images
2. Turn your folder into numbers
3. Train it and report the first result
4. Clean your data and train again
5. Test it on images from a new source
6. Put the demo on the web
7. Find out what your model looks at

What Your Model Looks At

Problem 7, the most important part of the assignment

- Guess what it uses, then change images on purpose to test the guess
- Cover a part, change the background, turn it grayscale, blur it
- You choose the guess and the change. Your agent only makes the change

Two Reports

- Short: one page of text and up to three pages of figures, written by you
- Long: as long as you like, written with your agent

The Short Report

- One page of text. Written by you, with no language model at any stage
- In Korean, if Korean is your first language
- What you chose and why, how it went, what you learned
- Template on Overleaf, link in the README

The Long Report

- Write it with your agent. That is what it is for
- Every number, every table, every figure, the whole checker output
- Template in `long_report/`, compiled on Overleaf

What You Are Given

- The image downloader
- The web page, with the webcam, upload and drawing already in it
- The export script and the checker
- A test for every function you have to write

What You Write

- `src/data.py`: read your folders, split them, prepare each photo
- `src/train.py`: load a pretrained ResNet, replace its last layer, train it
- `src/evaluate.py`: accuracy, confusion matrix, the worst mistakes
- Each function says exactly what goes in and what comes out

Getting Started

A GitHub account, and VS Code with Cline

Give Cline this request:

```
Set up MAS1004 Assignment 1 for me. Follow the section "First setup" in  
https://raw.githubusercontent.com/jdasam/mas1004-assignment1/main/AGENTS.md
```

- It makes your copy, installs everything, and runs the tests
- Copy made on Tuesday: use the update request in the README

How You Know It Worked

```
uv run pytest tests/test_data.py
```

- Red until you write it, green when it is right
- Paste the whole failure text to your agent, not your summary of it

The Plan Is Written Out for You

PLAN.md, step by step, each with a way to check it

- Give it to your agent one step at a time, not all at once
- Read it first. Writing one of these is part of what you are learning

Choosing your categories

Categories That Work

- Not one of ImageNet's 1,000 categories: see `imagenet_classes.txt`
- Things you can find new images of, outside your download
- Told apart by what is visible in the photo

Pick Something You Care About

- The grade is in the explanation, not the accuracy
- You will look at these images one by one in Problem 4
- Choose something you would still find interesting when the model gets it wrong

Downloading the Images

```
uv run python src/collect.py --classes "espresso cup,wine glass,paper cup" --n 150
```

- About 70 seconds per category
- Search phrases, not category names: change the phrase if a category comes back empty
- The cups are only an example. ImageNet already has cup, espresso and goblet

A Test Set from a New Source

At least 5 images per category that cannot be among your downloads

- For example your own photos, a friend's photos, frames from your own video
- Web images are fine if your download could not have found them
- Problem 5 measures the model on these, not on downloaded images

Git, GitHub, GitHub Pages

Why You Need These Today

- You hand in `links.txt`, with your demo and repository addresses
- Your demo has to be on the web, and GitHub puts it there for free

Git

A program that records every version of your files

- Every save is kept, with a message saying what changed
- You can go back to any of them

GitHub

A website that holds git repositories

The screenshot shows the GitHub interface for the repository `jdasm/mas1004`. The top navigation bar includes links for Platform, Solutions, Resources, Open Source, Enterprise, and Pricing, along with search, sign in, and sign up buttons. The repository header shows the name `jdasm/mas1004` as public, with notification, fork, and star buttons. Below the header, tabs for Code, Issues, Pull requests, Actions, Projects, Security and quality, and Insights are visible. The main content area displays the commit history with a table of recent changes, including updates to `data`, `demos`, and `slide_decks` folders, as well as configuration files like `.gitignore` and `README.md`. The right sidebar contains an 'About' section describing the repository as 'Data & AI at Sogang University', a 'Releases' section indicating no releases are published, and a 'Contributors' section listing `jdasm` and `claude`.

Commit	Message	Time
<code>851252c</code>	Update lecture 7 slide deck	last week
<code>...</code>	...	...
<code>...</code>	...	...

File	Message	Time
<code>data</code>	Add MNIST subset for lecture 7	last week
<code>demos</code>	Publish demo: lecture05-neuron-patch	2 weeks ago
<code>slide_decks</code>	Update lecture 7 slide deck	last week
<code>.gitignore</code>	2026 브랜치 시작: 이전 연도 과제와 라이브 코딩 자료를 빼고 ...	last month
<code>.nojekyll</code>	Publish demo: lecture03-regression-search	3 weeks ago
<code>README.md</code>	Add lecture 6 demo links	2 weeks ago

mas1004

Data & AI at Sogang University

Materials for the 2026 course. Lecture slides are in `slide_decks/`.

Interactive demos used in class (source in `demos/`, list of all demos at

About

Data & AI at Sogang University

Readme

Activity

10 stars

1 watching

1 fork

Report repository

Releases

No releases published

Contributors 2

- `jdasm` Dasaem Jeong
- `claude` Claude

Getting Your Own Copy

1. Make an account at github.com
2. Open github.com/jdasam/mas1004-assignment1
3. Press "Use this template", then "Create a new repository"
4. Keep it public

Or let Cline do 2 to 4 with the setup request

git clone

Copy your repository from GitHub onto your computer, once

```
git clone https://github.com/<you>/<your repo>.git
```

- You get a folder with every file and every past version

git add

Choose which changes go into the next version

```
git add src/data.py  
git add -A
```

- The first adds one file, the second every change in the folder
- Nothing is saved yet. The changes are only marked

git commit

Save the marked changes as one version, with a message

```
git commit -m "load_folder skips broken files"
```

- The version exists only on your computer
- The message says what changed, so you can find it later

git push

Send your new versions to GitHub

| git push

- Only now can anyone else see them, including me
- GitHub Pages rebuilds your demo from what you pushed

After Every Step

```
git add -A  
git commit -m "what I changed"  
git push
```

- Your coding agent can run all of these for you
- Push often. An unpushed commit is not handed in

GitHub Pages

Settings, then Pages, then Branch: main and Folder: /docs

- Your demo appears at `https://<you>.github.io/<your repo>/`
- It takes a minute or two the first time

Google Colab

A Computer in a Data Centre, for Free

- Nothing to install
- Notebook at `assignment1_colab.ipynb` in the repository
- Only for running: the code comes from your GitHub repository

Everything You Save There Is Deleted

So keep the work in your Google Drive instead

```
from google.colab import drive  
drive.mount('/content/drive')
```

- The notebook then works inside My Drive/mas1004/assignment1

Turning On the GPU

Runtime, Change runtime type, T4 GPU, Save

- Do it before you run anything. The session restarts
- Training a ResNet is much faster on a GPU